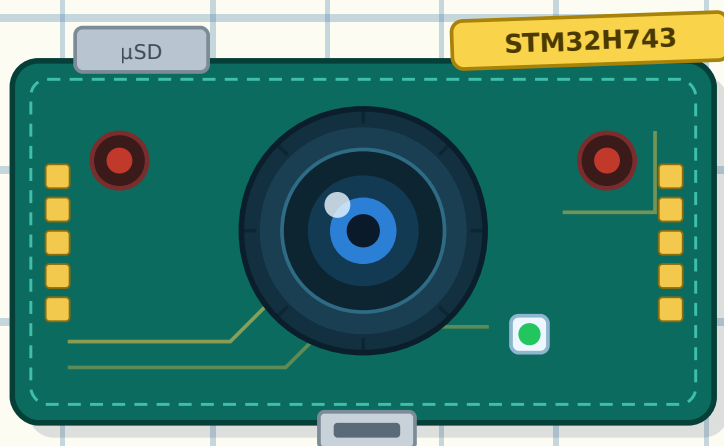


VISIÓN ARTIFICIAL · MICROPYTHON

OpenMV Cam H7 Plus

Manual ilustrado: qué es, qué hace, cómo programarla



Cortex-M7 · 480 MHz

Sensor OV5640 · 5 MP

32 MB SDRAM

MicroPython

Open source

Guía de referencia completa en español · Agosto 2026

◆ Qué vas a encontrar acá

1. **Qué es** la OpenMV Cam H7 Plus y de dónde salió
2. **Especificaciones** completas: cerebro, memoria, sensor, lente, pines, alimentación
3. **La familia OpenMV**: en qué se diferencia de la H7, M7, RT1062 y N6
4. **Software**: qué programa instalar, cómo conectarla y cómo actualizar el firmware
5. **Qué se puede hacer** con ella: 12 capacidades explicadas
6. **Cómo programarla**: 6 ejemplos de código listos para copiar
7. **Errores comunes**, trucos de FPS y manejo de memoria
8. **Recursos**: documentación, foro, GitHub y tutoriales

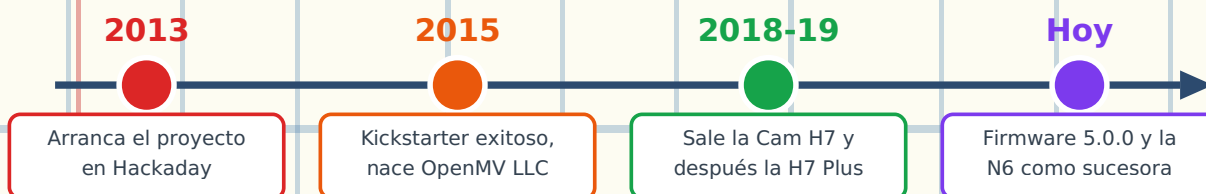
1 ¿Qué es la OpenMV Cam H7 Plus?

Es una **placa microcontroladora con cámara** que corre algoritmos de visión artificial por sí sola, sin necesitar una computadora. La idea del proyecto es simple y ambiciosa: ser "**el Arduino de la visión artificial**". En vez de programarla en C/C++ como un microcontrolador clásico, se programa en **Python** (MicroPython), lo que hace mucho más llevadero manejar las salidas complejas de un algoritmo de visión y, al mismo tiempo, controlar los pines de entrada/salida desde el mismo script.

Dicho de otra forma: le enchufás un cable USB, escribís 10 líneas de Python y la cámara ya te está diciendo "veo un objeto rojo en la coordenada (152, 87)" o "leí el QR y dice tal cosa". Después podés desconectarla de la PC y que siga funcionando sola, alimentada por batería, mandando esos datos a un Arduino, un motor o un servo.

► La historia en 4 hitos

Línea de tiempo del proyecto OpenMV



El proyecto lo fundaron Ibrahim Abdelkader y Kwabena Agyeman; buscaban un módulo de visión chico, expandible y de menos de 100 dólares.

► ¿Qué le agrega el "Plus"?

La H7 "a secas" y la H7 Plus comparten el mismo microcontrolador. La diferencia está en la memoria y el sensor:

OpenMV Cam H7 (normal)

- Sensor OV7725 (VGA)
- Solo 1 MB de SRAM interna
- Resolución limitada ($\approx 320 \times 240$)
- Se queda sin memoria rápido

OpenMV Cam H7 PLUS ✓

- Sensor **OV5640 de 5 MP**
- + **32 MB de SDRAM** externa
- + **32 MB de flash** QSPI externa
- Sin límites prácticos de resolución

◆ Precio, licencia y disponibilidad

- **Precio de lista:** 140 USD (SKU OMV-CAM-H7-PLUS-V1).
- **Licencia:** open source. Firmware, IDE, esquemáticos, PCB y hasta el archivo 3D de la carcasa están publicados en github.com/openmv. El firmware se basa en MicroPython (licencia MIT).
- **Ojo:** OpenMV **ya no la produce de forma regular** (solo para pedidos a granel). La página oficial la mantiene como documentación y recomienda la **OpenMV Cam N6** como sucesora. Igual la placa sigue 100% soportada por el software actual.
- **Dónde se consigue:** SparkFun, DigiKey, Mouser, Adafruit, Seeed, DFRobot, Antratek y algunos vendedores en Amazon (SingTown, el distribuidor oficial en China).

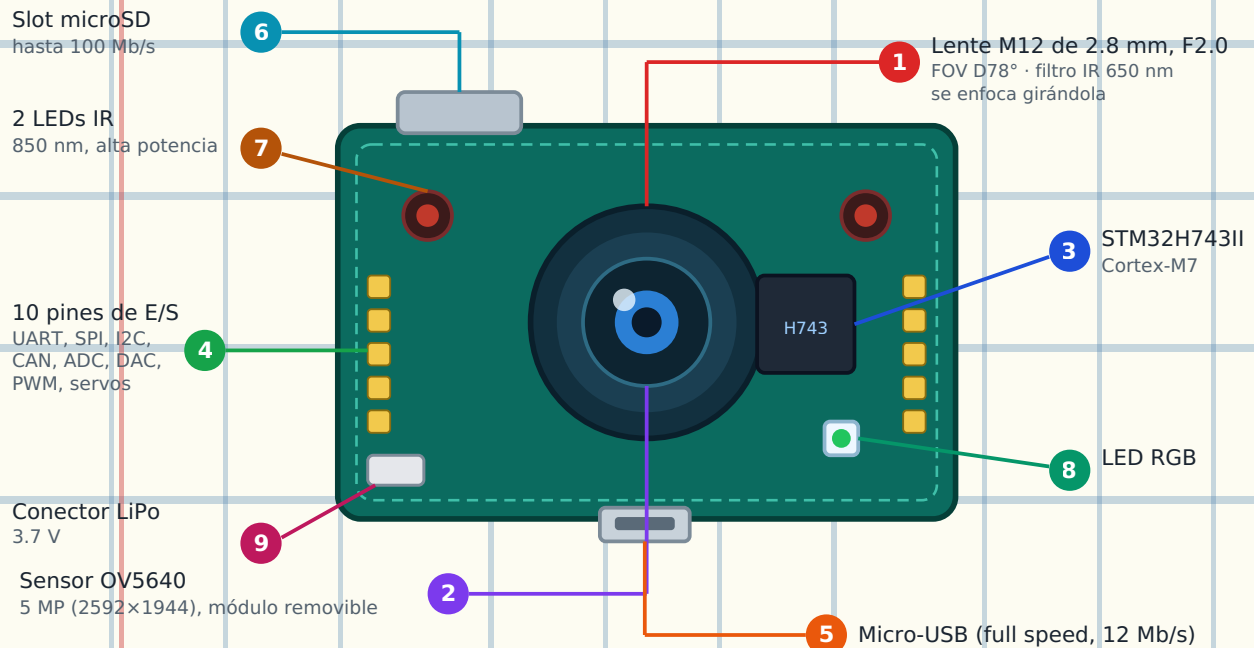
⚠ Tres cosas que NO tiene (mitos frecuentes)

No trae **micrófono MEMS**, no trae **IMU** (acelerómetro/giroscopio) ni **WiFi integrado**. Esas prestaciones están en la Arduino Nicla Vision y en las OpenMV AE3 y N6. Si querés WiFi en la H7 Plus, necesitás el **WiFi Shield** aparte. Si un tutorial da por sentado audio o movimiento onboard, seguro habla de otra placa.

2

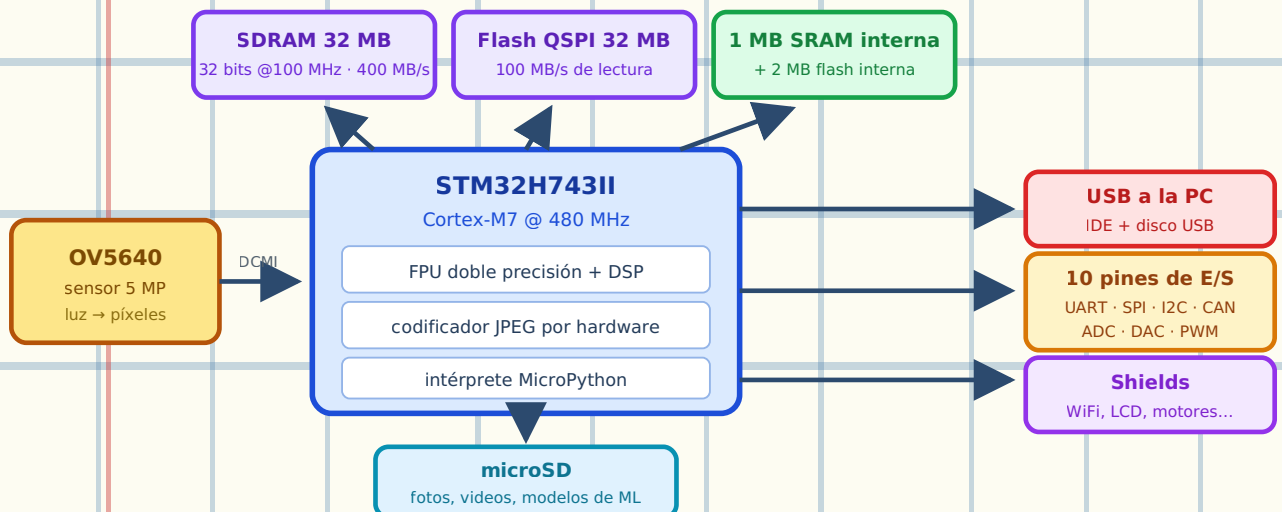
Especificaciones técnicas

► Anatomía de la placa



Vista superior esquemática. Medidas reales: 45 × 36 × 29 mm y unos 17 g de peso.

► Cómo fluye la información adentro



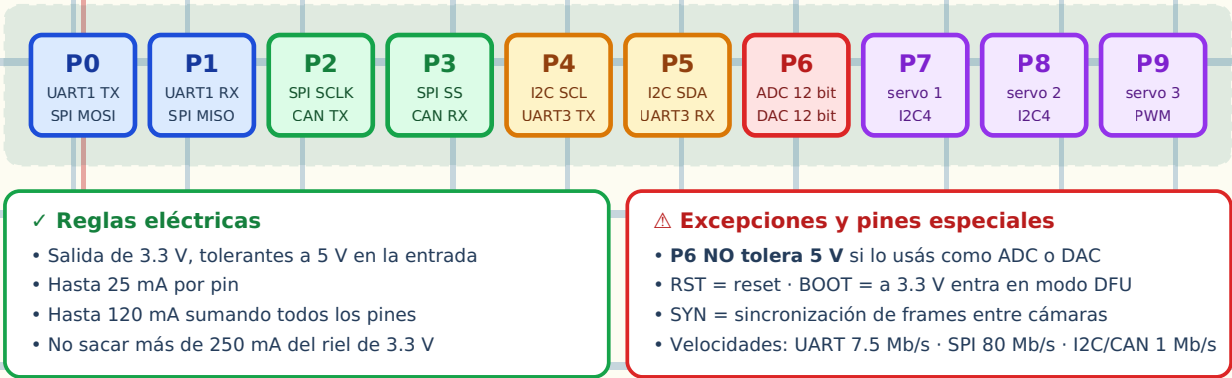
La luz entra por el lente → el sensor la convierte en píxeles → el M7 procesa en Python → salen DATOS, no imágenes

► Tabla de especificaciones

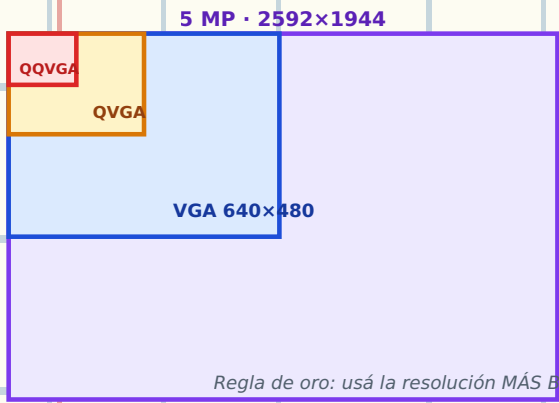
Componente	Detalle
Microcontrolador	STM32H743II · ARM Cortex-M7 de 32 bits · 480 MHz · 1027 DMIPS · CoreMark 2400 (la ficha oficial lo compara con una Raspberry Pi 2). FPU de doble precisión, set completo de instrucciones DSP y codificación JPEG por hardware.
Memoria RAM	1 MB de SRAM interna + 32 MB de SDRAM externa de 32 bits a 100 MHz (400 MB/s de ancho de banda, chip ISSI IS42S32800G).
Memoria flash	2 MB internos + 32 MB QSPI externos a 100 MHz en modo 4-bit DDR (100 MB/s de lectura, chip Winbond W25Q256JV).
Sensor de imagen	OmniVision OV5640 , formato 1/3", hasta 2592×1944 (5 MP) . Formatos de salida: Grayscale, RGB565, JPEG, Bayer RAW y YUV422 — todos hasta 5 MP. Módulo removible: se puede cambiar por un Global Shutter MT9V034 o una FLIR Lepton térmica. Soporta modo nocturno y autoenfoque (este último solo en el módulo OV5640 FPC).
Lente	2.8 mm, apertura F2.0, montura M12×0.5 . FOV real con el OV5640: D78.0° / H65.9° / V51.8°. Filtro IR de corte a 650 nm (se puede reemplazar por una lente sin filtro para ver infrarrojo, o por lentes de más/menos ángulo).
Alimentación	VIN de 3.6 a 5 V, o conector de batería LiPo de 3.7 V. Consumo: 140 mA en reposo, 230-240 mA activa a 3.3 V. Regulador conmutado de 1 A + LDO de bajo ruido para la parte analógica del sensor.
Almacenamiento	Socket microSD con lecturas/escrituras de hasta 100 Mb/s.
Extras onboard	1 LED RGB + 2 LEDs infrarrojos de 850 nm de alta potencia. Pin SYN para sincronizar frames entre varias cámaras.
Físico	45 × 36 × 29 mm · ~17 g · temperatura de operación -20 °C a 70 °C. Modelos 3D (.step) y carcasa imprimible (.stl) en GrabCAD.

► Los 10 pines de entrada/salida

Mapa de pines · todos con PWM e interrupciones



► **Resoluciones y velocidad**

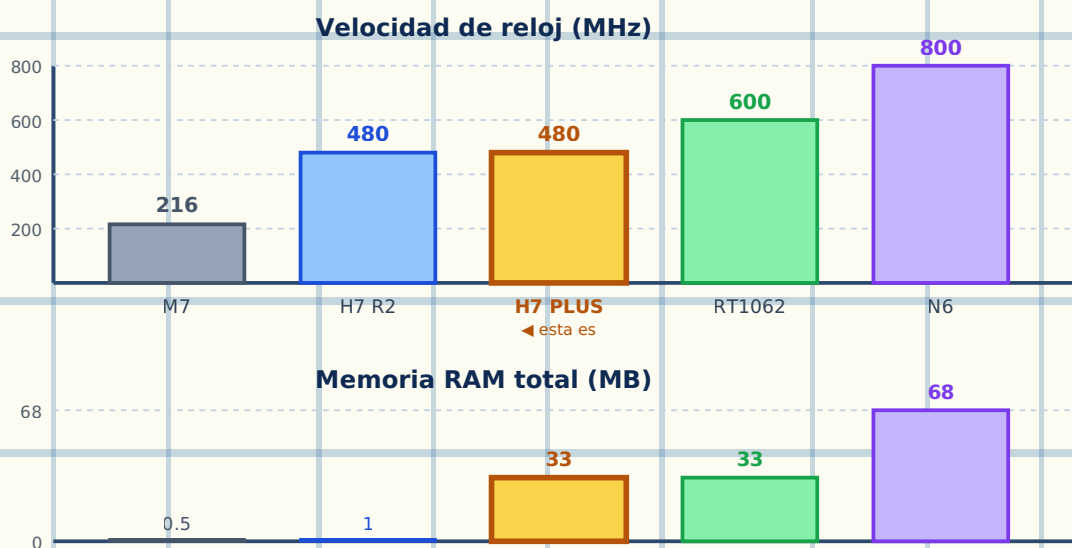


- ☐ **QQVGA 160×120**
La más liviana. Obligatoria para AprilTags y modelos de ML.
- ☐ **QVGA 320×240**
El punto dulce: la mayoría de los algoritmos corre a 25-50 FPS.
- ☐ **VGA 640×480**
Necesaria para códigos de barras 1D. Más lento y come más batería.
- ☐ **5 MP 2592×1944**
Solo para sacar fotos de alta calidad a la microSD, no para procesar.

3

La familia OpenMV: dónde encaja la H7 Plus

Modelo	Procesador	Reloj	RAM	Sensor	Extras
H7 Plus	STM32H743II (M7)	480 MHz	1 MB SRAM + 32 MB SDRAM	OV5640 · 5 MP	Sin WiFi · USB full speed · 140 USD
H7 R2	STM32H743 (M7)	480 MHz	1 MB SRAM (sin SDRAM)	menor resolución	La más barata; resolución limitada a $\approx 320 \times 240$
M7 (retirada)	STM32F765 (M7)	216 MHz	512 KB SRAM	OV7725	Modelo viejo; 320×240 RGB a 150 FPS
RT1062	i.MX RT1062 (M7)	600 MHz	1 MB SRAM + 32 MB SDRAM	OV5640 · 5 MP	WiFi/BLE, USB high speed, Ethernet, carga de batería, sueño de $30 \mu\text{A}$
N6 (sucesora)	STM32N657 (M55)	800 MHz	4.2 MB SRAM + 64 MB SDRAM	1 MP global shutter	NPU de 600 GOPS: corre YOLOv8/v11 a 30 FPS. Gigabit Ethernet, WiFi/BLE, H.264 por hardware



Fijate el salto de memoria: la H7 Plus tiene 33 veces más RAM que la H7 común. Eso es lo que le permite trabajar a 5 MP.

◆ ¿Cuándo conviene la H7 Plus y cuándo no?

Le conviene si: vas a usarla conectada por USB a una PC, o interconectada con otro microcontrolador por sus pines de 3.3 V. En esos casos rinde bárbaro y es más barata que las hermanas mayores. La documentación oficial aclara algo interesante: en ciertas situaciones la H7 Plus **rinde más que la RT1062**, porque sus buses de SRAM interna y SDRAM externa son más rápidos.

No le conviene si: necesitás WiFi o Ethernet integrados, carga de batería, consumo ultra bajo (microamperios) o una NPU para correr YOLO. Para eso están la RT1062 y la N6. Desplegar una H7 Plus "en el campo", lejos de una PC, es más engorroso.

4

El software: qué instalar y cómo empezar

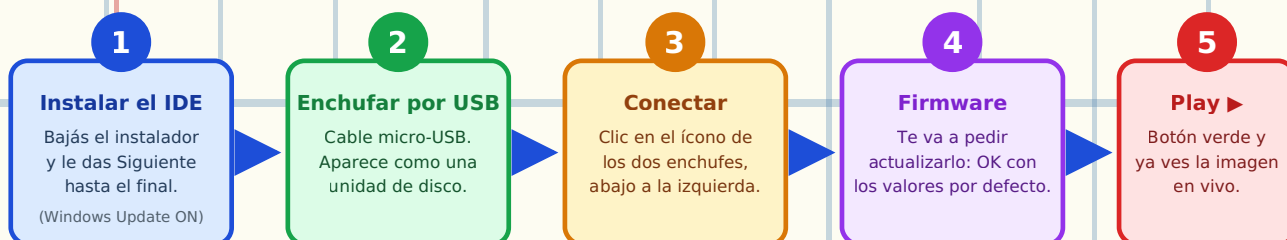
Todo se hace con **un solo programa**: el **OpenMV IDE**. Es gratuito, está hecho a medida para estas cámaras y viene con el firmware incluido, los ejemplos, las herramientas de calibración y el bootloader. No hace falta instalar Python, ni drivers por separado, ni nada más.

▼ Descarga oficial

<https://openmv.io/pages/download>

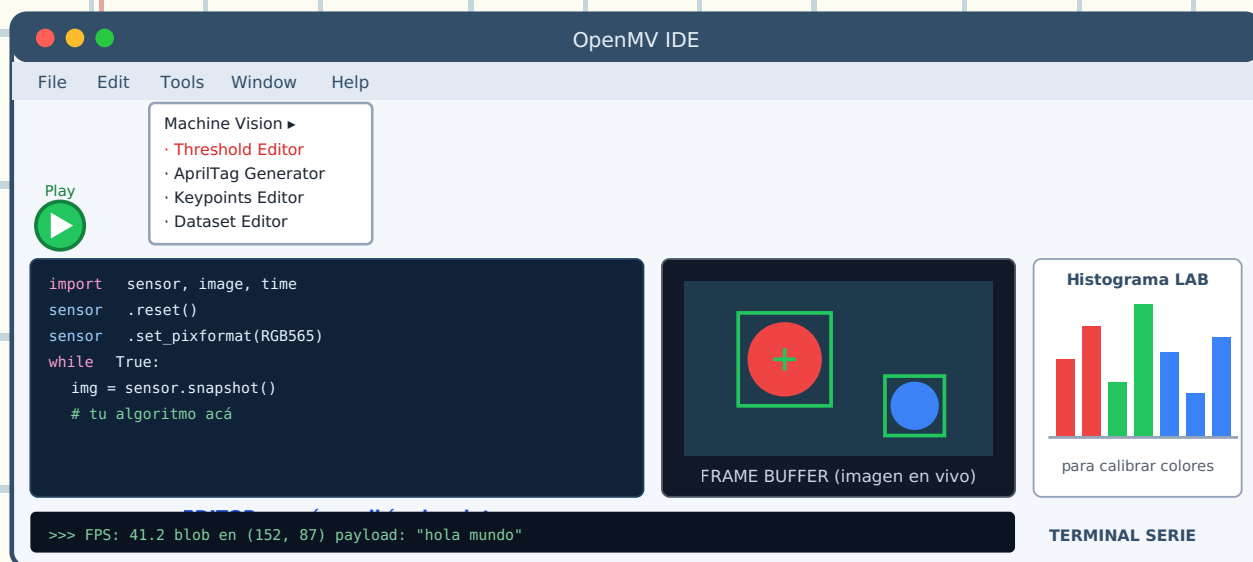
Versión actual: **OpenMV IDE 5.0.0** (trae el firmware 5.0.0, basado en MicroPython 1.28). Disponible para **Windows, macOS y Linux**. Las versiones anteriores están en github.com/openmv/openmv-ide/releases.

► Puesta en marcha, paso a paso



Si te pregunta si querés borrar el sistema de archivos interno, podés decir que NO para conservar lo que tenga guardado. Después de esto: girá la lente con los dedos hasta que la imagen se vea nítida. Viene desenfocada de fábrica.

► Cómo se ve el IDE por dentro



El editor tiene autocompletado y tooltips: si dejás el mouse sobre una función, te muestra su documentación.

◆ Que funcione sola, sin computadora

Guardá el script como `main.py` dentro de la unidad USB que monta la cámara (menú **Tools → Save open script to OpenMV Cam (as main.py)**). Al enchufarla a cualquier fuente de alimentación, la cámara ejecuta primero `boot.py` y después `main.py`, sola. **Bonus:** conectada al IDE la cámara corre a la mitad de velocidad, así que los FPS suben bastante cuando la desconectás.

► MicroPython: Python, pero en miniatura

La cámara corre **MicroPython 1.28**, una reimplementación de Python 3 pensada para microcontroladores. Escribís Python normal (bucles, funciones, listas, clases), pero con tres diferencias que conviene tener presentes:

- **La memoria es finita y chica.** Hay que administrarla; no podés cargar listas gigantes ni imágenes sin pensar.
- **Solo hay un subconjunto de la librería estándar.** No existe `pip install` de todo PyPI.
- **Lo pesado se delega a los módulos nativos.** Los módulos `sensor` e `image` están escritos en C dentro del firmware: cuando llamás a `find_blobs()`, el trabajo duro lo hace código compilado, no el intérprete Python. Por eso es rápido.

► Los módulos que vas a usar

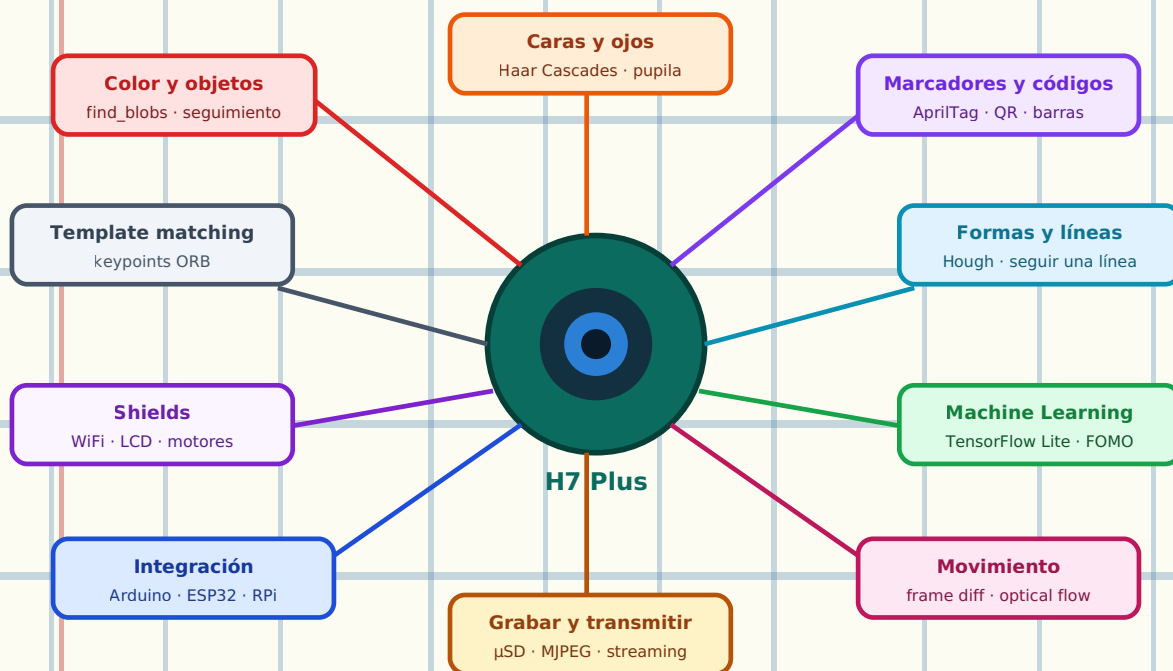
Módulo	Para qué sirve
sensor	Controlar la cámara: <code>reset()</code> , <code>set_pixformat()</code> , <code>set_framesize()</code> , <code>snapshot()</code> , <code>skip_frames()</code> , <code>set_auto_gain()</code> , <code>set_auto_whitebal()</code> , <code>set_windowing()</code> .
image	Todo el procesamiento: <code>find_blobs()</code> , <code>find_apriltags()</code> , <code>find_qrcodes()</code> , <code>find_barcodes()</code> , <code>find_datamatrices()</code> , <code>find_circles()</code> , <code>find_rects()</code> , <code>find_line_segments()</code> , <code>get_regression()</code> , <code>find_features()</code> , los <code>draw_*</code> y <code>lens_corr()</code> .
pyb	Hardware estilo pyboard: <code>LED</code> , <code>UART</code> , <code>SPI</code> , <code>I2C</code> , <code>DAC</code> , <code>Servo</code> , <code>Timer</code> , <code>RTC</code> .
machine	API estándar de MicroPython: <code>Pin</code> , <code>ADC</code> , <code>UART</code> , <code>SPI</code> , <code>I2C</code> , <code>CAN</code> .
time	<code>clock()</code> para medir FPS, <code>sleep_ms()</code> , <code>ticks</code> .
tf / ml	Correr modelos de TensorFlow Lite en la placa.
ulab	Cálculo numérico tipo NumPy.
network	Redes (requiere el WiFi Shield).
omv.rpc	Llamadas remotas para hablar con un Arduino, ESP32 o Raspberry Pi.

⚠ **Si la PC no la reconoce: recuperación por DFU**

1. Desenchufá y volvé a enchufar el cable. Nueve de cada diez veces alcanza.
2. Si el IDE se queda en "connecting", cancelá y aceptá cuando te ofrezca **recuperar el firmware por DFU**.
3. En Windows el DFU funciona directo. En Mac/Linux hay que haber corrido el `setup.sh` del paquete del IDE (permisos udev y grupo dialout).
4. Por hardware: se entra a modo DFU conectando el pin **BOOT a 3.3 V**.
5. **No uses una máquina virtual** para el bootloader: el handshake es sensible al tiempo y suele fallar.
6. Cargá solo el firmware de **tu modelo**. El IDE no verifica que coincida. Si cargás el equivocado la cámara **no se rompe**, solo se cuelga; se recupera cargando el correcto por DFU.

5

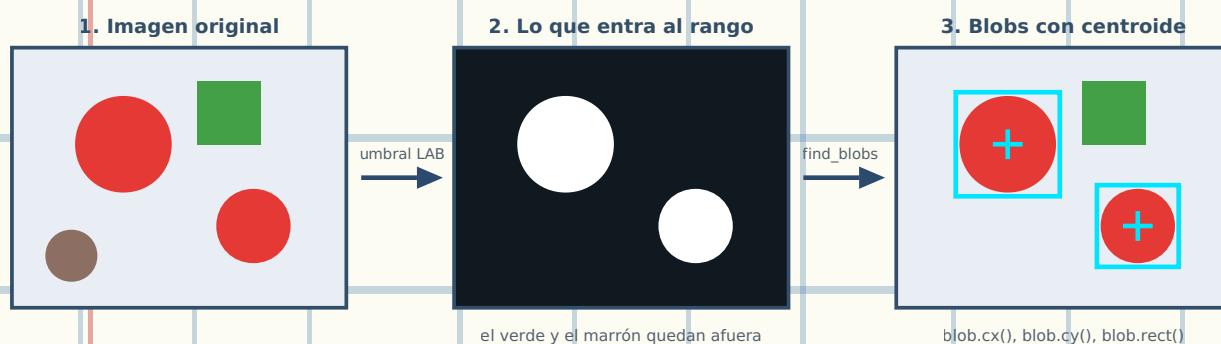
Qué se puede hacer con ella



Diez familias de capacidades. Todas se activan con una o dos líneas de Python.

► 1 · Detección de color (blob detection)

Es lo más usado. `img.find_blobs(thresholds)` busca "manchas" de color que caen dentro de un rango. Cada umbral es una tupla de **6 valores en espacio LAB**: (L min, L max, A min, A max, B min, B max). Podés pasar hasta 16 colores a la vez.



◆ Cómo sacar los números LAB sin volverte loco

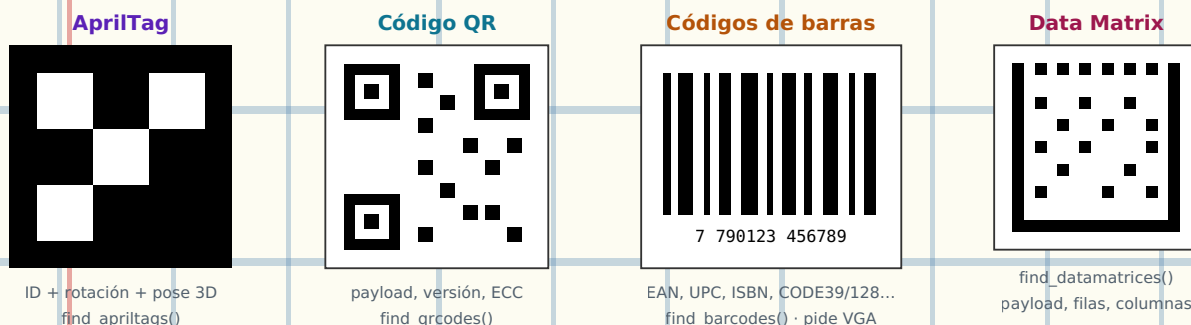
Andá a **Tools → Machine Vision → Threshold Editor**. Movés los 6 deslizadores y mirás la vista en blanco y negro: **lo blanco es lo que entra en el rango**. Cuando tu objeto queda blanco y el resto negro, copiás esos 6 números al script. Alternativa rápida: poné el histograma en modo LAB y dibujá un rectángulo con el mouse sobre el objeto en el frame buffer; el IDE te muestra los valores.

Parámetros útiles de `find_blobs`: `roi` (procesar solo una región), `pixels_threshold` y `area_threshold` (descartar manchitas de ruido), `merge=True` (fusionar blobs pegados), `x_stride/y_stride` (saltar píxeles para ir más rápido).

► 2 · Caras y ojos (Haar Cascades)

Se carga un clasificador con `image.HaarCascade("frontalface", stages=25)` y se buscan caras con `img.find_features(cascade, threshold=0.75, scale_factor=1.35)`. Tiene que ser en **GRAYSCALE**. Menos etapas (`stages`) = más rápido pero más falsos positivos. Corre sorprendentemente rápido porque usa imagen integral. También hay un cascade de ojos, y con `find_eye()` se localiza la pupila dentro del ojo: sirve, por ejemplo, para manejar un robot con la mirada.

► 3 · Marcadores y códigos



Los AprilTags son la joya de la corona. Son invariantes a traslación, rotación, escala, inclinación e iluminación, por lo que se detectan desde lejos y son mucho más robustos que seguir un color. Además de darte el ID, la cámara calcula la **pose 3D** del marcador — por eso se usan para realidad aumentada y para aterrizaje de precisión de drones. Familias soportadas: TAG16H5, TAG25H7, TAG25H9, TAG36H10, TAG36H11 (la de por defecto) y ARTOOLKIT. El propio IDE tiene un **generador de AprilTags** integrado que te arma un PNG listo para imprimir.

★ Dos trucos para los códigos

- Para **QR**, aplicá `img.lens_corr(1.8)`: corrige la distorsión de barril que mete la lente de 2.8 mm.
- Para **AprilTags**, bajá a **QQVGA** o te va a tirar error de memoria.
- Para **códigos de barras 1D** necesitás **GRAYSCALE en VGA**, y conviene usar `set_windowing((640, 80))` para procesar solo una franja y acelerar mucho.

► 4 · Formas, líneas y seguidor de línea

- `find_line_segments()` y `find_lines()` — transformada de Hough, para segmentos y líneas infinitas.
- `find_circles()` y `find_rects()` (este último reusa el detector de cuadriláteros de la librería AprilTag).
- **Seguidor de línea para robots:** `img.get_regression(thresholds)` ajusta una regresión lineal a los píxeles de la línea y devuelve `theta()` y `rho()`. Esos dos números se enchufan a un controlador PID y ya tenés un robot que sigue una línea. Si la línea es clara sobre piso gris, aplicá `histeq()` (ecualización de histograma) antes de binarizar.

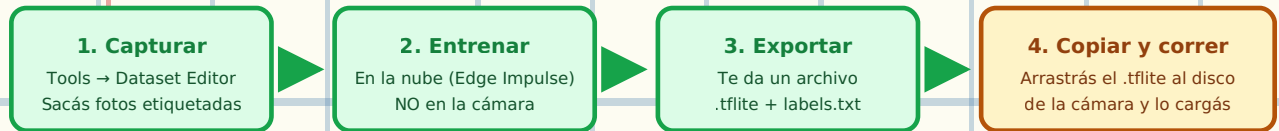
► 5 · Template matching, keypoints y optical flow

- **Template matching:** buscás una imagen guardada previamente dentro de la escena. Ideal para fiduciales de un PCB o dígitos de un display.
- **Keypoints (ORB):** descriptores de características para reconocer objetos aunque cambien de escala o rotación.
- **Optical flow:** mide cuánto se desplazó la escena entre dos frames. Es lo que usan los drones para mantenerse quietos sin GPS.

► 6 · Machine Learning en la placa

El firmware incluye **TensorFlow Lite for Microcontrollers**. El flujo típico es con **Edge Impulse**:

Flujo de trabajo de Machine Learning



El entrenamiento nunca ocurre en la cámara: ella solo hace la inferencia.

FOMO (Faster Objects, More Objects), el algoritmo de Edge Impulse, es el que mejor anda en placas tan chicas: permite contar objetos y ubicarlos en la imagen usando, según su documentación, hasta 30 veces menos memoria y procesamiento que MobileNet SSD o YOLOv5. En placas con el mismo Cortex-M7 a 480 MHz llega a unos 30 FPS con 245 KB de RAM.

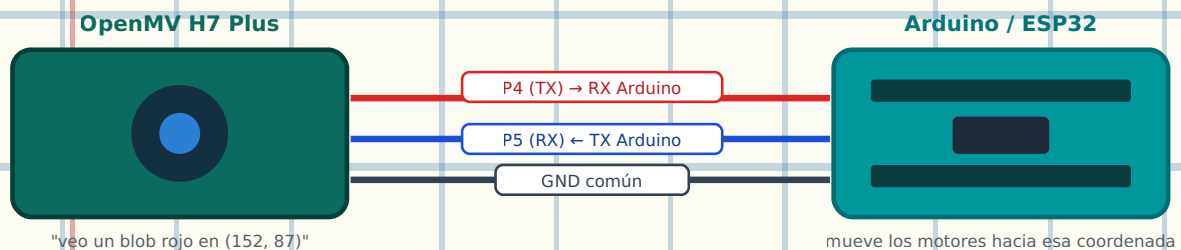
Sobre H7 Plus hay resultados publicados: un estudio en la revista *Sensors* (MDPI, 2025) reporta una velocidad promedio de inferencia de **55 FPS**, 3,2 veces más rápido que un YOLOv5 Tiny tradicional, con un pico de 137,7 KB de RAM. Y hay proyectos documentados de clasificación de variedades de arroz con FOMO (hasta 22,9 FPS), detección de barbijos y detección de personas.

► 7 · Movimiento, grabación y streaming

- **Frame differencing:** comparás el frame actual contra uno de referencia y detectás qué cambió. Base de cualquier sistema de seguridad casero.
- **Grabar a la microSD:** fotos en BMP/JPG/PPM/PGM y video en GIF o MJPEG. El script decide cuándo guardar (por ejemplo, solo cuando detecta movimiento).
- **Streaming:** con el WiFi Shield (módulo ATWINC1500, hasta 48 Mb/s) manda JPEG a un navegador y maneja sockets TCP/UDP con el módulo `network`. Por USB también puede funcionar como webcam con un firmware UVC alternativo.

► 8 · Como sensor de un Arduino, ESP32 o Raspberry Pi

Este es el caso de uso más común en robótica: la OpenMV hace la parte visual y le pasa **datos ya digeridos** (coordenadas, IDs, booleanos) al microcontrolador que maneja los motores.



Hay dos formas de hacerlo. La **simple**: mandar texto crudo por UART con `pyb.UART` (está el ejemplo en la sección de código). La **robusta**: usar la librería **RPC** que trae la cámara, con su contraparte `openmv-arduino-rpc` del lado de Arduino. Según el repositorio oficial, la RPC funciona sobre UART asíncrono (hasta 7,5 Mb/s), I2C y CAN (hasta 1 Mb/s) y SPI (hasta 20 Mb/s en la H7, y hasta 80 Mb/s con cables suficientemente cortos).

► 9 · Shields disponibles

Shield	Qué agrega
WiFi Shield	Módulo ATWINC1500, hasta 48 Mb/s. Streaming de video y sockets TCP/UDP.
LCD Shield	Pantalla TFT de 1.8" (128×160, RGB565) con retroiluminación. Para ver lo que ve la cámara sin PC.
Motor Shield	Regulador de 5 V + driver para dos motores DC de baja potencia. Robot completo en dos placas.
Pan-Tilt Shield	Control de 3 servos/ESC. Con esto armás la típica cámara que sigue una cara.
FLIR Lepton Adapter	Cámara térmica . Lepton 1 a 27 FPS (con 1/3 de frames repetidos); Lepton 3 a 9 FPS sin repetición.
Global Shutter Module	Sensor MT9V034 de 752×480 sin efecto rolling shutter: para objetos que se mueven rápido.
Proto Shield	Placa perforada para armar tu propio circuito encima.
TV / Wireless TV Shield	Salida de video compuesto.
RS422/RS485 Shield	Comunicación industrial a distancia.

◆ Dónde se usa esto en la vida real

Robótica: navegación, seguimiento de objetos, evasión de obstáculos, seguidores de línea. ·

Drones: optical flow para estabilidad y aterrizaje de precisión sobre AprilTags vía MAVLink. ·

Seguridad: cámaras que solo graban cuando detectan movimiento o una cara. · **Industria:** control de calidad, conteo de piezas, lectura de códigos en línea de producción. · **Agricultura:** clasificación de granos, detección de plagas. · **Educación:** es la plataforma ideal para un TP de visión artificial, porque el ciclo "escribo → veo el resultado" dura dos segundos.

6

Cómo programarla: 6 ejemplos listos

Todos estos scripts se pegan tal cual en el editor del IDE y se corren con el botón verde. La estructura siempre es la misma: **configurar el sensor una vez** arriba, y después un `while True` que saca una foto y la procesa.

① Hola Mundo — capturar imagen y medir FPS

```
import sensor, image, time

sensor.reset()                # inicializa el sensor
sensor.set_pixformat(sensor.RGB565) # color (o sensor.GRAYSCALE)
sensor.set_framesize(sensor.QVGA)  # 320x240
sensor.skip_frames(time=2000)      # deja que se estabilice 2 segundos

clock = time.clock()

while True:
    clock.tick()
    img = sensor.snapshot()        # toma la foto
    print(clock.fps())             # ojo: desconectada del IDE anda 2x más rápido
```

② Detección de blobs por color (rojo y verde)

```
import sensor, image, time

# (L min, L max, A min, A max, B min, B max) – sacalos del Threshold Editor
thresholds = [(30, 100, 15, 127, 15, 127), # índice 0 → rojo
              (30, 100, -64, -8, -32, 32)]  # índice 1 → verde

sensor.reset()
sensor.set_pixformat(sensor.RGB565)
sensor.set_framesize(sensor.QVGA)
sensor.skip_frames(time=2000)
sensor.set_auto_gain(False)      # CLAVE: si no, el color "baila"
sensor.set_auto_whitebal(False)  # CLAVE: idem

clock = time.clock()

while True:
    clock.tick()
    img = sensor.snapshot()
    for blob in img.find_blobs(thresholds,
                               pixels_threshold=100, # ignora manchitas
                               area_threshold=100,
                               merge=True):           # fusiona los pegados
        img.draw_rectangle(blob.rect())
        img.draw_cross(blob.cx(), blob.cy())
        print("objeto en:", blob.cx(), blob.cy())
    print(clock.fps())
```

③ Leer códigos QR

```
import sensor, image

sensor.reset()
sensor.set_pixformat(sensor.RGB565)
sensor.set_framesize(sensor.QVGA)
sensor.skip_frames(30)
sensor.set_auto_gain(False)

while True:
    img = sensor.snapshot()
    img.lens_corr(1.8) # corrige la distorsión de la lente 2.8mm
    for code in img.find_qrcodes():
        print(code.payload()) # el texto que tiene adentro el QR
```

④ Detectar AprilTags (con ID y rotación)

```
import sensor, image, time, math

sensor.reset()
sensor.set_pixformat(sensor.RGB565)
sensor.set_framesize(sensor.QQVGA) # baja resolución o se queda sin memoria
sensor.skip_frames(time=2000)
sensor.set_auto_gain(False)
sensor.set_auto_whitebal(False)

clock = time.clock()

while True:
    clock.tick()
    img = sensor.snapshot()
    for tag in img.find_apriltags(): # familia TAG36H11 por defecto
        img.draw_rectangle(tag.rect(), color=(255, 0, 0))
        img.draw_cross(tag.cx(), tag.cy(), color=(0, 255, 0))
        grados = 180 * tag.rotation() / math.pi
        print("ID:", tag.id(), "rotación:", grados)
    print(clock.fps())
```

⑤ Mandarle las coordenadas a un Arduino por UART

```
import sensor, image, time
from pyb import UART

uart = UART(3, 115200) # UART3 → TX en P4, RX en P5

sensor.reset()
sensor.set_pixformat(sensor.RGB565)
sensor.set_framesize(sensor.QVGA)
sensor.skip_frames(time=2000)
sensor.set_auto_gain(False)
sensor.set_auto_whitebal(False)

while True:
    img = sensor.snapshot()
    blobs = img.find_blobs([(30, 100, 15, 127, 15, 127)],
                           pixels_threshold=100, merge=True)

    if blobs:
        b = max(blobs, key=lambda x: x.pixels()) # el blob más grande
        uart.write("%d,%d\n" % (b.cx(), b.cy())) # "152,87\n"
    else:
        uart.write("-1,-1\n") # nada a la vista

# En el Arduino: Serial.readStringUntil('\n') y parseás los dos números.
# No te olvides de unir los GND de las dos placas.
```

⑥ Clasificar imágenes con un modelo TensorFlow Lite

```
import sensor, image, time, ml

sensor.reset()
sensor.set_pixformat(sensor.RGB565)
sensor.set_framesize(sensor.QVGA)
sensor.skip_frames(time=2000)

# el .tflite y el labels.txt se copian a la unidad USB de la cámara
model = ml.Model("trained.tflite", load_to_fb=True)
labels = [l.strip() for l in open("labels.txt")]

while True:
    img = sensor.snapshot()
    predicciones = model.predict([img])[0].flatten().tolist()
    for i, confianza in enumerate(predicciones):
        print(labels[i], ":", round(confianza, 2))
```

7

Errores comunes, FPS y memoria

► Buenas prácticas para que vuele

✓ Hacé esto

- Usá la **resolución más baja** con la que el algoritmo todavía funcione.
- Pasá a **GRAYSCALE** cuando el color no importe: más rápido y menos RAM.
- **Desconectá del IDE** para medir el FPS real.
- Apagá `auto_gain` y `auto_whitebal` para color y códigos estables.
- Limitá el área con `roi` y `set_windowing()`.
- Liberá memoria con `gc.collect()`.

✗ Evitá esto

- Procesar a 5 MP en vivo: eso es solo para sacar fotos a la microSD.
- Dejar la calidad JPEG en 100: usá 90 o menos para no desbordar el buffer.
- Llamar a `img.compress()` sin envolverlo en `try/except MemoryError`.
- Correr el bootloader desde una máquina virtual.
- Cargar el firmware de otro modelo de cámara.

► Tabla de rescate

Síntoma	Qué está pasando y cómo se arregla
La PC no la reconoce / el IDE queda en "connecting"	Desenchufá y volvé a enchufar; probá otro puerto USB y otro cable (ojo con los cables que son solo de carga). Si sigue, cancelá y aceptá la recuperación por DFU que ofrece el IDE. No lo hagas desde una máquina virtual.
"out of date - click here to upgrade"	El firmware es más viejo que el IDE. Hacé clic ahí, o andá a Tools → Run Bootloader . Tarda como un minuto y el LED parpadea.
MemoryError: Out of fast Frame Buffer Stack Memory	El algoritmo pide más RAM de la que hay a esa resolución. Bajá a QQVGA/ QVGA, pasá a GRAYSCALE, achicá el modelo de ML o meté un <code>gc.collect()</code> .
La imagen se ve borrosa o lechosa	Sacale la tapita al lente (sí, pasa) y girá la lente M12 con los dedos hasta que enfoque. Viene desenfocada de fábrica. En algunas unidades el módulo de cámara viene apenas torcido y hay que reacomodarlo.
El color detectado cambia solo	Están activos el auto gain y el auto white balance. Apagalos con <code>set_auto_gain(False)</code> y <code>set_auto_whitebal(False)</code> después del <code>skip_frames</code> .
Los FPS son bajísimos	Primero: ¿está conectada al IDE? Ahí corre a la mitad. Después: bajá resolución, usá ROI, pasá a escala de grises y aumentá los <code>stride</code> .
No lee los QR / códigos de barras	QR: aplicá <code>lens_corr(1.8)</code> y acercá el código. Barras: necesitás GRAYSCALE en VGA sí o sí, y buena iluminación pareja.
Cargué el firmware equivocado	No se rompió: solo se colgó. Entrá en modo DFU (pin BOOT a 3.3 V) y cargá el firmware correcto de tu modelo.

◆ Oficiales

- **openmv.io** — sitio principal
- **docs.openmv.io** — documentación completa (tutorial, librerías, quickref por placa, shields, sensores)
- **openmv.io/pages/download** — descargar el IDE
- **forums.openmv.io** — el foro; los desarrolladores responden ahí
- **github.com/openmv** — firmware, IDE, RPC para Arduino, apriltag, tensorflow-lib, proyectos

◆ Para aprender

- **book.openmv.cc/en/** — tutorial paso a paso muy completo (de SingTown)
- **DigiKey Maker** — "Getting started" y el proyecto de pan-tilt que sigue caras
- **Arduino Docs** — blob detection, filtro de caras Haar, Edge Impulse
- **Hackster.io** — decenas de proyectos con FOMO y TinyML
- Los **ejemplos del propio IDE**: File → Examples. Empezá por *02-Color-Tracking* y *16-Codes*.

◆ Datasheets si querés ir a fondo

STM32H743II y su manual de referencia RM0433 (en st.com) · OV5640 (en la página del producto) · SDRAM ISSI IS42S32800G · Flash Winbond W25Q256JV · Quick reference específico: docs.openmv.io/openmvcam/quickref/openmv-cam-h7-plus.html

★ Plan sugerido para el primer día

1. Instalar el OpenMV IDE, conectar la cámara y aceptar la actualización de firmware.
2. Correr el "Hola Mundo" y **girar la lente** hasta que la imagen se vea nítida.
3. Abrir *File* → *Examples* → *02-Color-Tracking* y calibrar tu propio color con el Threshold Editor bajo la iluminación real donde vas a usarla.
4. Imprimir un AprilTag desde el generador del IDE y probar el ejemplo ④.
5. Guardar tu script como `main.py`, desconectar del IDE y ver cómo sube el FPS.

Manual armado a partir de la documentación oficial de OpenMV (openmv.io, docs.openmv.io, foro y GitHub), datasheets de ST y OmniVision, y tutoriales de DigiKey, Arduino y Edge Impulse.

Las cifras de FPS son aproximadas: dependen de la resolución, la iluminación y la complejidad del algoritmo.